

Minos API Documentation & JSON Data

Model Formats

The Minos project follows an MVC design pattern. The back end is organized into “drivers” (business logic) and “tables” (ODM using MongoEngine) with an API layer (in `/routes/api.py`) that the front end interacts with.

This documentation explains:

- **ID Format Convention:**
 - **GET responses:** All `_id` fields are returned in MongoDB Extended JSON format.
For example:
 - `"_id": {"$oid": "67c445a097f2f27870fc89ee"}`
 - **POST, PUT, DELETE requests:** Supply any `_id` as a string (the string representation of the ObjectID).
 - **Field Types:** Detailed for each model below.
 - **API Endpoints:** Including expected request/response JSON formats.
 - **Treatment POST Examples:** Complete JSON examples for creating treatments of type *Treatment*, *Regimen*, and *Alternative*.
-

1. Data Model Details & Field Types

1.1 Characteristic Model

- **`_id`:**
 - Type: ObjectID in Extended JSON (GET) / string (in requests)
- **`char_type`:**
 - Type: string
 - Example: "Primary Indication", "Population", "Biomarker"

- **name:**
 - Type: string
 - Example: "Lung Cancer", "Iran", "KRAS G12C"

GET JSON Example:

```
{
  "_id": {"$oid": "60a7eb5a9c8e4b0015d8a125"},
  "type": "Primary Indication",
  "name": "Lung Cancer"
}
```

1.2 Drug Model

- **_id:**
 - Type: ObjectID in Extended JSON (GET) / string (in requests)
- **name:**
 - Type: string
 - Example: "Carboplatin"
- **strength:**
 - Type: integer
 - Example: 450
- **unit:**
 - Type: string
 - Allowed values: "mg", "g", "ng", "mcg", "IU"

GET JSON Example:

```
{
  "_id": {"$oid": "60a7eb5a9c8e4b0015d8a127"},
  "name": "Carboplatin",
  "strength": 450,
  "unit": "mg"
}
```

1.3 Treatment Model

- **_id:**
 - Type: ObjectID in Extended JSON (GET) / string (in requests)
- **name:**
 - Type: string
- **type:**
 - Type: string
 - Allowed values: "Treatment", "Regimen", "Alternative"
- **regimen:** (*Optional – used when type is "Regimen"*)
 - Type: object
 - Contains:
 - **drugs:** array of objects, each with:
 - **drug:** an object with:
 - **_id:** ObjectID (GET) / string (in requests)
 - **name:** string
 - **strength:** integer
 - **unit:** string (one of "mg", "g", "ng", "mcg", "IU")
 - **annual_patient_con:** integer
- **alternatives:** (*Optional – used when type is "Alternative"*)
 - Type: array of objects, each with:
 - **regimen:** object (same structure as above)
 - **ratio:** float
- **treatment_hash:**
 - Type: string (automatically computed on the server)

GET JSON Example:

```
{
  "_id": {"$oid": "60a7eb5a9c8e4b0015d8a128"},
  "name": "First-line 1 Treatment",
```

```

"type": "Regimen",
"regimen": {
  "drugs": [
    {
      "drug": {
        "_id": {"$oid": "60a7eb5a9c8e4b0015d8a127"},
        "name": "Carboplatin",
        "strength": 450,
        "unit": "mg"
      },
      "annual_patient_con": 100
    }
  ],
  "alternatives": [],
  "treatment_hash": "computed_hash_value"
}

```

1.4 Followup Model

- **_id:**
 - Type: ObjectID in Extended JSON (GET) / string (in requests)
- **name:**
 - Type: string
- **overall_survival:**
 - Type: float
- **patient_id:**
 - Type: ObjectID in Extended JSON (GET) / string (in requests)
- **parent_id:**
 - Type: ObjectID in Extended JSON (GET) / string (in requests)

GET JSON Example:

```

{
  "_id": {"$oid": "60a7eb5a9c8e4b0015d8a131"},

```

```
"name": "Followup for Patient X",
"overall_survival": 0.75,
"patient_id": {"$oid": "60a7eb5a9c8e4b0015d8a130"},
"parent_id": {"$oid": "60a7eb5a9c8e4b0015d8a135"}
}
```

1.5 PatientTree Model & Node Structure

- **PatientTree Fields:**

- **_id:** ObjectID in Extended JSON (GET) / string (in requests)
- **tree:** object (the root Node)
- **tree_hash:** string

- **Node Fields:**

- **_id:** ObjectID in Extended JSON (GET) / string (in requests)
- **rate:** float
- **size:** float
- **node_type:** string
 - Allowed values: "characteristic", "treatment", "followup"
- **parent_id:** ObjectID in Extended JSON (GET) or null
- **Embedded Payload:** (Only one present based on node_type)
 - For "characteristic" nodes:
characteristic_data:
 - **_id:** ObjectID in Extended JSON (GET) / string (in requests)
 - **char_type:** string
 - **name:** string
 - For "treatment" nodes:
treatment_data:
 - Follows the Treatment model structure.
 - For "followup" nodes:
followup_data:
 - Follows the Followup model structure.

- **children:** array of Node objects (recursive structure)

GET Node JSON Example:

```
{
  "_id": {"$oid": "60a7eb5a9c8e4b0015d8a135"},
  "rate": 1.0,
  "size": 90000000,
  "node_type": "characteristic",
  "parent_id": null,
  "characteristic_data": {
    "_id": {"$oid": "60a7eb5a9c8e4b0015d8a125"},
    "char_type": "Population",
    "name": "Iran"
  },
  "children": [
    {
      "_id": {"$oid": "60a7eb5a9c8e4b0015d8a136"},
      "rate": 0.0001,
      "size": 9000,
      "node_type": "characteristic",
      "parent_id": {"$oid": "60a7eb5a9c8e4b0015d8a135"},
      "characteristic_data": {
        "_id": {"$oid": "60a7eb5a9c8e4b0015d8a137"},
        "char_type": "Primary Indication",
        "name": "Lung Cancer"
      },
      "children": []
    }
  ]
}
```

GET PatientTree JSON Example:

```
{
  "_id": {"$oid": "60a7eb5a9c8e4b0015d8a130"},
  "tree": { /* Node JSON structure as shown above */ },
  "tree_hash": "computed_hash_value"
```

}

2. API Endpoints Overview

Characteristics Endpoints

- **GET** /api/characteristics

Retrieves all characteristics.

Response Example:

- [
 - {
 - "_id": {"\$oid": "60a7eb5a9c8e4b0015d8a125"},
 - "type": "Primary Indication",
 - "name": "Lung Cancer"
 - },
 - {
 - "_id": {"\$oid": "60a7eb5a9c8e4b0015d8a126"},
 - "type": "Population",
 - "name": "Iran"
 - }
-]

- **POST** /api/characteristics

Creates a new characteristic.

Expected Request JSON:

- {
 - "type": "Primary Indication",
 - "name": "Lung Cancer"
- }

Response Example:

```
{  
  "id": "60a7eb5a9c8e4b0015d8a125"
```

```
}
```

- **PUT** /api/characteristics/<char_id>

Updates an existing characteristic.

Expected Request JSON (any subset):

- {
- "type": "Updated Type",
- "name": "Updated Name"
- }

Response Example:

```
{  
  "message": "Characteristic updated"  
}
```

- **DELETE** /api/characteristics/<char_id>

Deletes a characteristic.

Response Example:

- {
- "message": "Characteristic deleted"
- }

Drugs Endpoints

- **GET** /api/drugs

Retrieves all drugs.

Response Example:

- [
- {
- "_id": {"\$oid": "60a7eb5a9c8e4b0015d8a127"},
- "name": "Carboplatin",
- "strength": 450,
- "unit": "mg"
- }
-]

- }
-]
- **POST** /api/drugs

Creates a new drug.

Expected Request JSON:

- {
- "name": "Carboplatin",
- "strength": 450,
- "unit": "mg"
- }

Response Example:

```
{
  "id": "60a7eb5a9c8e4b0015d8a127"
}
```

- **PUT** /api/drugs/<drug_id>
- Updates a drug.*

Expected Request JSON (any subset):

- {
- "name": "Updated Drug Name",
- "strength": 500,
- "unit": "mg"
- }

Response Example:

```
{
  "message": "Drug updated"
}
```

- **DELETE** /api/drugs/<drug_id>
- Deletes a drug.*

Response Example:

- {
 - "message": "Drug deleted"
 - }
-

Treatments Endpoints

- **GET** /api/treatments

Retrieves all treatments.

Response Example:

- [
- {
- "_id": {"\$oid": "60a7eb5a9c8e4b0015d8a128"},
- "name": "First-line 1 Treatment",
- "type": "Regimen",
- "regimen": {
- "drugs": [
- {
- "drug": {
- "_id": {"\$oid": "60a7eb5a9c8e4b0015d8a127"},
- "name": "Carboplatin",
- "strength": 450,
- "unit": "mg"
- },
- "annual_patient_con": 100
- }
-]
- },
- "alternatives": [],
- "treatment_hash": "computed_hash_value"
- }
-]

- **POST** /api/treatments

Creates a new treatment.

Note: When sending a POST request, supply any `_id` values as strings.
(See the Treatment POST examples below for specific payloads.)

- **PUT** `/api/treatments/<treatment_id>`

Updates a treatment.

Expected Request JSON (any subset):

- {
- "message": "Updated Treatment Name",
- "regimen": { /* updated regimen JSON if applicable */ }
- }

Response Example:

```
{
  "message": "Treatment updated"
}
```

- **DELETE** `/api/treatments/<treatment_id>`

Deletes a treatment.

Response Example:

- {
- "message": "Treatment deleted"
- }

Patients (PatientTree) Endpoints

- **GET** `/api/patients`

Retrieves all patient tree documents.

Response Example:

- [
- {
- "`_id`": { "`$oid`": "60a7eb5a9c8e4b0015d8a130" },
- "`tree`": { /* Node JSON structure as shown above */ },
- "`tree_hash`": "computed_hash_value"
- }

- }
-]
- **POST** /api/patients

Creates a new patient tree using a single root node.

Expected Request JSON (legacy style):

- {
- "node": {
- "node_type": "characteristic",
- "rate": 1.0,
- "size": 90000000,
- "parent_id": null,
- "characteristic_data": {
- "_id": "60a7eb5a9c8e4b0015d8a125",
- "char_type": "Population",
- "name": "Iran"
- },
- "children": []
- }
- }

Response Example:

```
{
  "id": "60a7eb5a9c8e4b0015d8a130"
}
```

- **PUT** /api/patients/<patient_id>

Updates an existing patient tree.

Expected Request JSON (optional fields):

- {
- "tree": { /* full updated Node JSON structure */ },
- "size": 85000000
- }

Response Example:

```
{
  "message": "Patient updated"
}
```

- **DELETE** /api/patients/<patient_id>

Deletes the entire patient tree document.

Response Example:

```
• {
•   "message": "Patient deleted"
• }
```

Node Management Endpoints

- **POST** /api/patients/<patient_id>/add_node

Adds a new node to an existing patient tree.

Expected Request JSON:

```
• {
•   "parent_node_id": "60a7eb5a9c8e4b0015d8a135",
•   "node": {
•     "node_type": "treatment",
•     "rate": 0.8,
•     "size": 50000,
•     "treatment_data": {
•       "_id": "60a7eb5a9c8e4b0015d8a128",
•       "name": "First-line 1 Treatment",
•       "type": "Regimen",
•       "regimen": {
•         "drugs": [
•           {
•             "drug": {
•               "_id": "60a7eb5a9c8e4b0015d8a127",
•               "name": "Carboplatin",
•               "strength": 450,
•               "unit": "mg"
•             }
•           },
•           {
•             "drug": {
•               "_id": "60a7eb5a9c8e4b0015d8a128",
•               "name": "Cisplatin",
•               "strength": 450,
•               "unit": "mg"
•             }
•           }
•         ]
•       }
•     }
•   }
• }
```

- "annual_patient_con": 100
- }
-]
- }
- }
- }
- }

Response Example:

```
{
  "message": "Node added successfully",
  "tree_hash": "new_computed_hash"
}
```

- **PUT** /api/patients/<patient_id>/node/<node_id>
Updates an existing node in the patient tree.

Expected Request JSON (any subset):

- {
- "rate": 0.9,
- "size": 60000,
- "characteristic_data": {
- "id": "60a7eb5a9c8e4b0015d8a125",
- "char_type": "Biomarker",
- "name": "KRAS"
- }
- }

Response Example:

```
{
  "message": "Node updated successfully",
  "tree_hash": "new_computed_hash"
}
```

- **DELETE** /api/patients/<patient_id>/node/<node_id>
Deletes a node (splicing its children into the parent).

Response Example:

- {
 - "message": "Node deleted successfully",
 - "tree_hash": "new_computed_hash"
 - }
-

Followups Endpoints

- **GET** /api/followups
Retrieves all followup records.

Response Example:

- [
- {
- "_id": {"\$oid": "60a7eb5a9c8e4b0015d8a131"},
- "name": "Followup for Patient X",
- "overall_survival": 0.75,
- "patient_id": {"\$oid": "60a7eb5a9c8e4b0015d8a130"},
- "parent_id": {"\$oid": "60a7eb5a9c8e4b0015d8a135"}
- }
-]

- **POST** /api/followups
Creates a new followup record.

Expected Request JSON:

- {
- "name": "Followup for Patient X",
- "overall_survival": 0.75,
- "patient_id": "60a7eb5a9c8e4b0015d8a130",
- "parent_id": "60a7eb5a9c8e4b0015d8a135"
- }

Response Example:

```
{
  "id": "60a7eb5a9c8e4b0015d8a131"
}
```

- **PUT** /api/followups/<followup_id>

Updates a followup record.

Expected Request JSON (any subset):

- {
- "name": "Updated Followup Name",
- "overall_survival": 0.8
- }

Response Example:

```
{
  "message": "Followup updated"
}
```

- **DELETE** /api/followups/<followup_id>

Deletes a followup record.

Response Example:

- {
- "message": "Followup deleted"
- }

3. Treatment POST Request Examples

Below are complete examples of JSON payloads to create new treatments using a POST request to /api/treatments.

3.1 Simple Treatment

For a treatment that does not include any additional regimen or alternative information.

Request JSON:

```
{
  "name": "Single Agent Therapy",
  "type": "Treatment"
}
```

3.2 Regimen Treatment

For a regimen treatment that includes one or more drugs.

Request JSON:

```
{
  "name": "Carboplatin + Paclitaxel Regimen",
  "type": "Regimen",
  "regimen": {
    "drugs": [
      {
        "drug": {
          "_id": "60a7eb5a9c8e4b0015d8a127",
          "name": "Carboplatin",
          "strength": 450,
          "unit": "mg"
        },
        "annual_patient_con": 100
      },
      {
        "drug": {
          "_id": "60a7eb5a9c8e4b0015d8a128",
          "name": "Paclitaxel",
          "strength": 100,
          "unit": "mg"
        },
        "annual_patient_con": 100
      }
    ]
  }
}
```

```
}  
}
```

3.3 Alternative Treatment

For an alternative treatment where multiple regimens are proposed with associated ratios.

Request JSON:

```
{  
  "name": "Alternative Regimens for NSCLC",  
  "type": "Alternative",  
  "alternatives": [  
    {  
      "regimen": {  
        "drugs": [  
          {  
            "drug": {  
              "_id": "60a7eb5a9c8e4b0015d8a127",  
              "name": "Carboplatin",  
              "strength": 450,  
              "unit": "mg"  
            },  
            },  
          "annual_patient_con": 100  
        }  
      ],  
    },  
    "ratio": 0.75  
  },  
  {  
    "regimen": {  
      "drugs": [  
        {  
          "drug": {  
            "_id": "60a7eb5a9c8e4b0015d8a129",  
            "name": "Gemcitabine",  
            "strength": 1000,  
            },  
          },  
        ],  
      },  
    },  
  ],  
}
```

```
        "unit": "mg"
      },
      "annual_patient_con": 120
    }
  ]
},
"ratio": 0.25
}
]
```

4. Front-End Integration Notes

- **ObjectIDs:**
 - **GET Responses:** `_id` fields are returned in MongoDB Extended JSON format, for example:
 - `"_id": {"$oid": "67c445a097f2f27870fc89ee"}`
 - **Other Requests:** Supply `_id` values as plain strings.

- **Data Types:**

Follow the types outlined above when constructing requests and parsing responses.

- **HTTP Methods & Status Codes:**

Standard REST conventions apply:

- **GET** for retrieval.
- **POST** for creation.
- **PUT** for updates.
- **DELETE** for removal.

On error, responses include an `"error"` key with details.

- **Payload Structure:**

Use the provided examples to ensure consistency. For partial updates (via PUT), include only the fields that need to be changed.

This documentation should serve as a comprehensive guide for integrating the front end with the Minos back end. Adjust or expand the examples as needed to fit your project's specific requirements.